

Architecture Analysis

1. How are the layers organised and what are the dependency rules between them?

noCommerce is a monolithic ASP.NET Core application. Everything runs in one process. There are no microservices, no message queues, no separate deployable units. Plugins extend the same process rather than running alongside it.

That said, the code is split into well-defined layers with a strict top-down dependency rule:



Each layer only knows about the layers below it. Core sits at the bottom and has no dependencies on anything else. Data depends only on Core. Services depends on Core and Data. And so on up the stack.

Plugins (there are about 31 of them) sit alongside the web layer. Most reference Nop.Web directly to get everything transitively. They don't bypass the hierarchy by referencing Core or Data on their own.

— All dependency injection registration happens in one place: the startup class inside Nop.Web.Framework. That's where every service interface gets bound to its implementation, and where event consumers are discovered via assembly scanning.

2. How does noCommerce handle events internally, what is IEventPublisher and how is it used?

IEventPublisher is the internal event bus. It's how noCommerce decouples "something happened" from "here's what should happen next."

The idea is simple: when a service does something meaningful (like placing an order), it publishes an event. Other parts of the system, called consumers, react to that event independently. The service that published the event doesn't know or care who's listening.

On the publishing side, any service can call something like `_eventPublisher.PublishAsync(new OrderPlacedEvent(order))`. On the consuming side, any class that implements `IConsumer<T>`

gets picked up automatically at startup through assembly scanning. No manual registration needed.

When an event is published, the system finds all consumers for that event type, resolves them from DI, and runs them one by one. It's worth noting:

- Events are not queued or durable. They execute inline, as part of the same request.
- Consumers run sequentially, not in parallel.
- If a consumer throws an exception, it gets logged and swallowed. The rest of the consumers still run, and the caller never sees the error.

There are two kinds of events in practice:

- **Entity lifecycle events** that fire automatically whenever something is inserted, updated, or deleted in the database. These are mostly used for cache invalidation.
- **Domain events** like `OrderPlacedEvent`, `OrderPaidEvent`, `ShipmentSentEvent`, etc. These fire at explicit business moments and trigger things like email notifications, reward points, and third-party integrations.

The net effect is that a service like `OrderProcessingService` doesn't need to know about emails, caching, or analytics. It just publishes events and moves on. This is also how plugins hook into the system without modifying core code.

3. Where does the code make it easy to add observability, and where does it make it hard?

What works in our favour

The event system gives us free hooks. Since every meaningful business action publishes an event, we can create consumers that record metrics or start trace spans without touching any existing service code. Just implement `IConsumer<OrderPlacedEvent>` and the system picks it up.

ASP.NET Core already has built-in tracing support. The framework's `DiagnosticSource` and `Activity` APIs mean that HTTP-level telemetry (request timing, status codes, routes) comes for free once we plug in the OpenTelemetry ASP.NET Core package. No code changes at all for this layer.

The service methods are well-structured. The order flow, for example, is broken down into clear steps: validate, process payment, save, adjust inventory. Each step is its own method, which maps naturally to child spans in a trace.

What makes it harder

There's no tracing infrastructure at all. nopCommerce doesn't use `ActivitySource` or `System.Diagnostics.Activity` anywhere. So while the HTTP layer gets traced automatically, everything below it is a black box. To see what's happening inside the service layer, we have to add tracing manually to specific methods.

Services call each other directly. There's no middleware, no interceptor pattern, no pipeline between service calls. When `OrderProcessingService` calls `PaymentService`, it's a direct method call through an injected interface. That means there's no way to automatically wrap those calls with spans. Each one has to be instrumented by hand.

Payment goes through plugins, which are opaque. The payment service delegates to a dynamically loaded plugin for the actual gateway call. Trace context flows through automatically (via AsyncLocal), but the plugin itself has no instrumentation. So we can see that payment was called and how long it took overall, but not what happened inside the plugin.

4. What would you need to change structurally to instrument it properly, and is that change worth making?

Add a shared ActivitySource to the service layer

The most important change. We need a single **ActivitySource** that the key service methods can use to create trace spans. Without this, traces stop at the HTTP boundary and there's no visibility into the business logic.

The change is small: one static declaration, and then a few lines wrapping each key method. It follows the same pattern ASP.NET Core uses internally. Worth doing, minimal risk.

Register OpenTelemetry in the startup

Add the OTel NuGet packages and configure tracing and metrics in the application startup. This is purely additive, no existing code gets modified. It's just wiring: tell OTel which sources to listen to and where to export.

Add a PII sanitisation processor

The order flow handles sensitive data: customer names, emails, addresses, payment details. Rather than trying to remember which fields to exclude at every instrumentation point (which is error-prone), a better approach is a centralised processor that runs before export and strips anything matching a known PII pattern.

One class, registered once in the pipeline, and it acts as the single enforcement point for the privacy strategy. Worth it for correctness and maintainability.

— What's not worth doing

Wrapping every service interface with a tracing decorator would give automatic span creation for all method calls, but it would generate a lot of noise. Most service methods aren't interesting from an observability perspective. Targeted instrumentation on just the key methods is cleaner and gives better signal-to-noise.

Similarly, instrumenting the EventPublisher itself to create spans for each consumer invocation is possible but not necessary for this scope. The spans around the methods that publish events already give enough visibility.

Summary

The minimum changes to make this work:

1. A shared **ActivitySource** with spans in about five key service methods

2. OpenTelemetry SDK registration at startup
3. A PII sanitisation processor to keep sensitive data out of traces

These are small, surgical changes. They don't touch business logic and they follow standard .NET observability patterns.